

State

状态设计优化

概述

优化 dp 时，不止可以从转移过程入手，加速转移。有时，也可以从状态定义入手，通过改变设计状态的方式实现复杂度上的优化。

令人比较头疼的是，这类优化大多不具有通用性，即不能很套路地应用于多个题目中。因此，下文将从具体例题出发，力求提供思路上的启发，希望对读者有一定帮助。

例 1

▼ 题面

给定两个长度分别为 n 且仅由小写字母构成的字符串，求 s 的最长公共子序列。

朴素的解法

您一眼秒了它，这不是板子吗？

定义状态 $f[i][j]$ 为 s 的前 i 位与 t 的前 j 位最长公共子序列，则有

上述做法的时间复杂度，无法通过本题。

更优的解法

我们仔细一想，发现了一个性质：最终答案不会超过 $\min(n, m)$ 。

我们又仔细一想，发现 LCS 满足贪心的性质。

更改状态定义 $f[i]$ 为与 t 前 i 位的最长公共子序列长度为 j 的最短前缀长度（即将朴素做法的答案与第一维状态对调）

可以通过预处理 t 的每一位的下一个 $t[i]$ 的出现位置进行 f 的顺推转移。

复杂度，可以通过本题。

例 2

▼ 题面

给定一个 n 个点的无权有向图，判断该图是否存在哈密顿回路。

朴素的解法

看到数据范围，我们考虑状压。

设 $f[S][i]$ 表示从点 i 出发，仅经过点集 S 中的点能否到达点 i 。记 g 为原图的邻接矩阵。则有

时间复杂度，写得好看或许能过，但是并不优美。

更优的解法

上面的状态设计中，每个 值只代表一个 `bool` 值，这让我们觉得有些浪费。

我们可以考虑对于每个状态 将 压成一个 `int`，发现我们可以将邻接矩阵同样压缩后进行 转移。

时间复杂度，可以通过这道题，其中 为 `int` 的位数。

本页面最近更新：2023/2/18 07:57:07，[更新历史](#)

发现错误？想一起完善？[在 GitHub 上编辑此页！](#)

本页面贡献者：[Marcythm](#), [partychicken](#), [StudyingFather](#), [Xeonacid](#), [Enter-tainer](#), [hqztrue](#), [Mascros](#), [Mout-sea](#), [Tiphereth-A](#)

本页面的全部内容[在 CC BY-SA 4.0 和 SATA 协议之条款下](#)提供，附加条款亦可能应用